SIMATS ENGINEERING

CSA0823 – PYTHON PROGRAMMING

ASSIGNMENT – 5 (CO5)

DATA ACQUISITION, CLEANING, ANALYSIS AND AI APPLICATIONS

Submitted by:

Name: D.S. Tharuniya Priya

Register Number: 192511317

Programme: B.E. Computer Science and Engineering

Year: II Year

Submitted to:

Faculty Name: Hariramachandran

Department of Computer Science and Engineering

SIMATS Engineering

Github Repo Link : https://github.com/tharu-ui/CSA0823---Python-Programing/tree/main/C05%20Assignment%205

TABLE OF CONTENTS

S. No.	Topic
1	Part A – Design and Analysis
1.1	A1 – Data Acquisition Design
1.2	A2 – Data Cleaning and Normalisation
1.3	A3 – Data Analysis and Visualisation
1.4	A4 – Debugging and Profiling
1.5	A5 – AI-Based Applications
2	Part B – Implementation and Testing
2.1	B1 – Data Cleaning, Database Storage and Visualisation
2.2	B2 – Adversarial Testing, SQL Injection and Sentiment Analysis
2.3	B3 – Performance Comparison of Data Processing Methods
2.4	B4 – Sentiment Analysis, Summarisation and Database Indexing

PART A – DESIGN AND ANALYSIS

A1 – DATA ACQUISITION DESIGN

1. Introduction

Data acquisition is the process of collecting data from different sources and storing it in a structured format for further processing and analysis.

In this case study, air-quality readings are collected from 12 monitoring stations. The collected data contains information such as date, hour, PM2.5, PM10, nitrogen oxide (NO), and temperature.

The collected data is stored in CSV files. However, the dataset may contain missing values, duplicate records, invalid measurements, inconsistent temperature units, and incorrectly formatted entries.

Therefore, a systematic data acquisition process is required to inspect, validate, clean, and store the collected readings.

2. Structure of the Input Dataset

The input CSV file contains the following fields:

Column Name	Description	Data Type
station_id	Unique identification of the monitoring station	Integer
station_name	Name of the monitoring station	String
date	Date on which the reading was recorded	Date
hour	Hour of the reading	Integer
PM2.5	Concentration of fine particulate matter	Float
PM10	Concentration of coarse particulate matter	Float
NO	Nitrogen oxide concentration	Float
temperature	Recorded temperature	Float
temperature_unit	Unit of temperature, either Celsius or Fahrenheit	String

3. Possible Data Quality Issues

The dataset may contain several data quality problems that can affect the accuracy of the analysis.

Data Quality Issue	Example	Proposed Solution
Missing values	PM2.5 field contains NA	Replace using an appropriate statistical method
Negative readings	PM10 = -25	Identify as invalid and reject or replace
Impossible values	PM2.5 = 1500	Apply validation limits
Duplicate records	Same station, date and hour repeated	Remove duplicate entries
Inconsistent temperature units	Some stations report Fahrenheit	Convert all values into Celsius
Incorrect data types	Date stored as an invalid string	Validate and convert into a standard date format
Incorrect delimiters	Semicolon used instead of comma	Validate CSV structure before processing

4. Proposed SQLite Database Design

SQLite is selected as the database because it is lightweight, serverless, and suitable for storing structured air-quality readings.

The proposed database design separates station information from individual air-quality readings. This reduces repeated station details and maintains data consistency through primary key and foreign key constraints. During implementation, the database structure is created according to the tables and columns used by the Python program.

Two tables are proposed: Stations and Readings.

Table 1: Stations

Column	Data Type	Constraint	Description
station_id	INTEGER	PRIMARY KEY	Unique station identifier
station_name	TEXT	UNIQUE, NOT NULL	Name of the station
temperature_unit	TEXT	NOT NULL	Original temperature unit

Table 2: Readings

Column	Data Type	Constraint	Description
reading_id	INTEGER	PRIMARY KEY	Unique reading identifier
station_id	INTEGER	FOREIGN KEY	Reference to the station
reading_date	TEXT	NOT NULL	Date of measurement
reading_hour	INTEGER	NOT NULL	Hour of measurement
pm25	REAL	-	PM2.5 concentration
pm10	REAL	-	PM10 concentration
no_value	REAL	-	Nitrogen oxide concentration
temperature_c	REAL	-	Temperature converted to Celsius

A composite UNIQUE constraint on station_id, reading_date, and reading_hour can prevent duplicate readings for the same station and time.

Relationship between the tables:

Stations (1) -------- (Many) Readings

One monitoring station can have multiple air-quality readings. The station_id column in the Readings table acts as a foreign key referencing the Stations table.

5. Data Acquisition and Storage Process

The proposed data acquisition process consists of the following steps:

1. Read the CSV file from the input directory.

2. Inspect the column names and data types.
3. Validate the presence of all required columns.
4. Identify missing and invalid values.
5. Check for duplicate records.
6. Convert temperature readings into Celsius.
7. Apply the required cleaning procedures.
8. Insert the cleaned records into the SQLite database.
9. Execute SQL queries to retrieve useful information.
10. Generate summary reports for further analysis.

6. SQL Queries for Data Retrieval

Query 1: Calculate the average PM2.5 concentration for each station during October.

```
SELECT s.station_name, AVG(r.pm25)
FROM Stations s
JOIN Readings r ON s.station_id = r.station_id
WHERE r.reading_date >= '2026-10-01'
AND r.reading_date < '2026-11-01'
GROUP BY s.station_name;
```

This query calculates the average PM2.5 concentration for every monitoring station during October.

Query 2: Identify the five stations with the highest average PM2.5 concentration.

```
SELECT s.station_name, AVG(r.pm25) AS avg_pm25
FROM Stations s
JOIN Readings r ON s.station_id = r.station_id
GROUP BY s.station_name
ORDER BY avg_pm25 DESC
```

LIMIT 5;

This query helps identify the stations experiencing the highest average particulate pollution.

Query 3: Identify the hours with the highest average PM2.5 concentration.

SELECT reading_hour, AVG(pm25) AS avg_pm25

FROM Readings

GROUP BY reading_hour

ORDER BY avg_pm25 DESC;

This query groups the readings by hour and identifies the hours during which pollution levels are highest.

7. Comparison Between CSV Processing and SQL Storage

Feature	**CSV / DataFrame Processing**	**SQLite Database**
Storage	Stores data in files	Stores structured records in tables
Retrieval	Requires reading and scanning records	Supports SQL-based retrieval
Filtering	Performed using loops or DataFrame conditions	Performed using WHERE conditions
Aggregation	Performed using functions such as groupby	Performed using GROUP BY
Indexing	Requires additional data structures	Supports database indexes
Relationships	No built-in relational constraints	Supports primary and foreign keys
Suitable Use	Small datasets and file exchange	Structured storage and repeated queries

8. Complexity Analysis

Let n represent the total number of records.

- Reading and scanning a complete CSV file requires O(n) time.

- Filtering all records using a DataFrame generally requires O(n) time.
- A database query using an appropriate B-tree index can retrieve matching records in approximately O(log n) lookup time, excluding the time required to return all matching records.
- A complete SQL aggregation still needs to process the relevant records and generally requires O(n) processing time, with additional overhead for grouping and sorting.

Conclusion:

CSV files are useful for collecting and exchanging data. However, SQLite is more suitable for structured storage, repeated queries, indexing, and maintaining relationships between stations and readings.

A2 – DATA CLEANING AND NORMALISATION

1. Introduction

Data cleaning is the process of identifying and correcting errors, inconsistencies, missing values, and duplicate records in a dataset.

In air-quality monitoring, inaccurate measurements can lead to incorrect pollution reports. Therefore, cleaning must be performed before analysis.

2. Handling Missing Values

Missing values may occur when sensors fail to record readings or when data transmission is interrupted.

The following methods can be used:

Method	Description	Advantages	Limitations
Deletion	Remove records containing missing values	Simple to implement	May result in data loss
Forward Fill	Use the previous valid reading	Useful for time-series data	May not reflect actual changes
Median Imputation	Replace missing values with the median	Less affected by extreme values	May reduce natural variation
Mean Imputation	Replace missing values with the mean	Simple statistical approach	Sensitive to extreme values

Selected Method: Median Imputation

Median imputation is selected for recoverable missing or invalid numerical readings because it is less affected by extreme values than the mean. During implementation, recoverable missing values are replaced using the selected imputation procedure. Values that cannot be reliably recovered or fail validation are excluded from the cleaned dataset. This prevents unreliable measurements from affecting the analysis.

3. Handling Invalid and Impossible Values

The dataset must be checked against acceptable measurement limits.

The following validation rules are proposed:

1. PM2.5 values must not be negative.
2. PM10 values must not be negative.
3. Nitrogen oxide readings must not be negative.
4. Temperature values must fall within a reasonable physical range.
5. Extremely high values must be investigated before being accepted.
6. Records with impossible values should be rejected or corrected using a justified method.

For the implementation, readings greater than 999 are excluded as specified in the assignment.

4. Duplicate Record Removal

Duplicate records occur when the same monitoring station reports the same reading more than once for an identical date and hour.

To identify duplicates, the following fields are considered together:

- Station ID
- Reading date
- Reading hour

If two records contain the same combination of these fields, only one valid record is retained.

5. Temperature Unit Conversion

Some stations report temperature in Fahrenheit, while others report temperature in Celsius.

For consistent analysis, all temperature readings are converted into Celsius.

Fahrenheit to Celsius conversion:

Celsius = (Fahrenheit - 32) × 5 / 9

Example:

Given Fahrenheit = 77°F

Celsius = (77 - 32) × 5 / 9

Celsius = 45 × 5 / 9

Celsius = 25°C

Therefore, 77°F is equivalent to 25°C.

6. Data Normalisation

Normalisation is used to bring numerical values into a comparable scale.

Two commonly used techniques are Min-Max Normalisation and Z-Score Normalisation.

A. Min-Max Normalisation

Min-Max Normalisation converts values into a range between 0 and 1.

Formula:

Normalised value = (Original value - Minimum value) / (Maximum value - Minimum value)

Example:

Consider the values 48, 95, and 121.

Minimum value = 48

Maximum value = 121

For 48:

(48 - 48) / (121 - 48) = 0

For 95:

(95 - 48) / (121 - 48) = 47 / 73 = 0.644

For 121:

(121 - 48) / (121 - 48) = 1

The normalised values are:

Original Value	Normalised Value
48	0.000
95	0.644
121	1.000

B. Z-Score Normalisation

Z-Score Normalisation measures how far a value is from the mean in terms of standard deviations.

Formula:

Z-score = (Original value - Mean) / Standard deviation

A positive Z-score indicates that the value is above the mean, while a negative Z-score indicates that it is below the mean.

7. Standardisation of Data Formats

The following standards are adopted:

Data Field	Standard Format
Date	YYYY-MM-DD
Time	24-hour format
Temperature	Celsius
PM2.5	Numeric value
PM10	Numeric value
NO	Numeric value
Missing values	Consistently identified and handled

8. Cleaning Algorithm – Pseudocode

```
BEGIN

  INPUT air_quality_dataset
```

```
CHECK whether all required columns exist

FOR each record in dataset

    VALIDATE station identification

    VALIDATE date and hour

    IF any required reading is missing
        REPLACE recoverable missing value using median
    END IF

    IF any reading is negative or impossible
        REJECT or correct the record
    END IF

    IF any reading is greater than 999
        SKIP the record
    END IF

    CHECK for duplicate station, date and hour

    IF duplicate record exists
        REMOVE duplicate
    END IF

    IF temperature unit is Fahrenheit
        CONVERT temperature into Celsius
    END IF

    NORMALISE selected numerical values
```

END FOR

SAVE cleaned dataset

END

9. Comparison of Alternative Cleaning Methods

Method	**Time Complexity**	**Advantages**	**Disadvantages**
Dropping missing records	O(n)	Simple and fast	Can lose useful data
Forward filling	O(n)	Preserves time-series continuity	May repeat outdated values
Median imputation	O(n log n) for sorting-based median calculation	Robust against extreme values	Requires a representative median
Duplicate removal	O(n) average using a hash-based approach	Prevents repeated measurements	Requires a reliable duplicate key

Conclusion:

A combination of validation, duplicate removal, median imputation, temperature conversion, and normalisation provides a reliable cleaning process. These steps improve the quality and consistency of the air-quality dataset.

A3 – DATA ANALYSIS AND VISUALISATION

1. Introduction

Data analysis is used to identify patterns, trends, relationships, and unusual observations in collected data.

Visualisation makes complex datasets easier to understand and helps identify pollution patterns across different monitoring stations and time periods.

2. Selection of Suitable Charts

Different chart types are selected depending on the purpose of the analysis.

Analysis Requirement	**Selected Chart**	**Reason**
Monthly PM2.5 trend	Line Chart	Shows changes over time
Comparison between stations	Bar Chart	Makes station-wise comparison easier
Relationship between temperature and PM2.5	Scatter Plot	Shows possible correlation
Identification of worst pollution hours	Bar Chart	Compares pollution levels across hours
Distribution of PM2.5 readings	Histogram	Shows frequency distribution
Identification of outliers	Box Plot	Displays spread and unusual values

3. Monthly PM2.5 Trend

A line chart is used to display the average PM2.5 concentration across different months.

The date is placed on the X-axis, and average PM2.5 concentration is placed on the Y-axis.

This chart helps identify:

- Increasing pollution trends.
- Decreasing pollution trends.

- Seasonal variations.
- Unusual changes in pollution levels.

4. Comparison Between Monitoring Stations

A bar chart is used to compare average PM2.5 values across the 12 monitoring stations.

The station names are displayed on the X-axis, and average PM2.5 concentration is displayed on the Y-axis.

This chart helps identify stations with consistently higher pollution levels.

5. Relationship Between Temperature and PM2.5

A scatter plot is used to examine the relationship between temperature and PM2.5 concentration.

Temperature is displayed on the X-axis, while PM2.5 is displayed on the Y-axis.

Each point represents an individual observation.

The scatter plot helps determine whether temperature and particulate pollution show a possible relationship.

6. Identification of Worst Pollution Hours

A bar chart is used to compare average PM2.5 readings at different hours of the day.

The hour is displayed on the X-axis, and average PM2.5 concentration is displayed on the Y-axis.

This analysis helps identify the time periods during which pollution levels are highest.

7. Daily Summary Statistics

A daily summary is generated for each monitoring station.

The summary contains the following information:

Field	**Description**
Station ID	Unique station identifier
Date	Date of observation

Minimum PM2.5	Lowest PM2.5 reading for the day
Maximum PM2.5	Highest PM2.5 reading for the day
Average PM2.5	Mean PM2.5 reading for the day
Reading Count	Number of valid readings

Calculation of Daily Statistics

Minimum PM2.5 = Lowest valid PM2.5 reading of the day

Maximum PM2.5 = Highest valid PM2.5 reading of the day

Average PM2.5 = Sum of valid PM2.5 readings / Number of valid readings

Reading Count = Total number of valid readings

8. Daily Summary Algorithm – Pseudocode

```
BEGIN

    INPUT cleaned_dataset

    GROUP records by station and date

    FOR each station and date group

        INITIALISE minimum value
        INITIALISE maximum value
        INITIALISE total value
        INITIALISE reading count

        FOR each reading in the group

            UPDATE minimum value
            UPDATE maximum value
```

ADD reading to total value
INCREMENT reading count

END FOR

CALCULATE average value

STORE station, date, minimum,
maximum, average and count

END FOR

SAVE daily summary

END

9. Python Processing Versus SQL Aggregation

Feature	**Python Loops**	**SQL GROUP BY**
Processing	Iterates through records	Database performs grouping
Memory	May require storing intermediate data	Can process data within the database
Repeated Queries	Requires repeated processing	Queries can be reused
Performance	Depends on implementation	Can benefit from database optimisation
Complexity	O(n)	Generally O(n) for scanning and grouping, with additional overhead where sorting is required

Conclusion:

Charts and summary statistics provide meaningful insights into pollution patterns. Line charts are suitable for time-based trends, bar charts for comparisons, and scatter plots for relationships between numerical variables. SQL aggregation is useful for generating daily summaries efficiently.

A4 – DEBUGGING AND PROFILING

1. Introduction

Debugging is the process of identifying and correcting errors in a program. Profiling is used to measure program performance and identify operations that consume significant execution time or resources.

In an air-quality monitoring system, debugging and profiling help ensure that data is processed correctly and efficiently.

2. Debugging Techniques

A. Print Statements

Print statements display intermediate values during program execution.

They help identify:

- Incorrect variable values.
- Missing records.
- Unexpected results.
- Errors during data processing.

B. Assertions

Assertions are used to verify important conditions in a program.

Examples of conditions include:

- PM2.5 values must not be negative.
- Station identifiers must be valid.
- Required columns must exist.
- Database insertion must not create duplicate readings.

C. Logging

Logging provides a structured way to record program events.

Different logging levels include:

Logging Level	Purpose
DEBUG	Detailed information for debugging
INFO	Normal program execution information
WARNING	Unexpected situations that do not stop execution
ERROR	Errors that prevent an operation
CRITICAL	Serious failures affecting the system

D. Python Debugger (pdb)

The Python debugger allows execution to pause at specific points.

It helps developers:

- Inspect variable values.
- Execute the program step by step.
- Identify the location of errors.
- Understand program flow.

3. Profiling Techniques

Profiling helps measure the performance of a program.

A. time.perf_counter()

This function measures elapsed time with high-resolution timing.

It can be used to compare the execution time of different processing methods.

B. timeit

The timeit module measures the execution time of small code segments.

It is useful for comparing alternative implementations.

C. cProfile

The cProfile module provides detailed information about program execution.

It reports:

- Number of function calls.
- Total execution time.
- Time spent in individual functions.
- Cumulative execution time.

4. Example Profiling Procedure – Pseudocode

BEGIN

RECORD start time

EXECUTE data processing operation

RECORD end time

CALCULATE elapsed time

DISPLAY execution time

RUN program profiler

IDENTIFY functions consuming more time

ANALYSE performance results

END

5. Comparison of Debugging Methods

Method	**Advantages**	**Limitations**
Print statements	Simple and easy to use	Can clutter program output

Assertions	Checks important conditions	May be disabled in optimised execution
Logging	Maintains structured records	Requires configuration
Debugger	Allows step-by-step inspection	May take additional time
Profiler	Identifies performance bottlenecks	Does not automatically correct errors

Conclusion:

Debugging ensures the correctness of data processing, while profiling helps improve execution efficiency. Logging and profiling tools are especially useful in systems that process large volumes of air-quality readings.

A5 – AI-BASED APPLICATIONS

1. Introduction

Artificial Intelligence can be used to analyse citizen complaints, identify public concerns, and provide automated responses.

In the air-quality monitoring system, citizens may submit complaints about pollution, smoke, waste disposal, traffic, and other environmental problems.

Natural Language Processing techniques can be applied to understand these complaints.

2. Sentiment Analysis

Sentiment analysis identifies the emotional tone of a text.

A complaint can be classified into three categories:

Sentiment	Meaning	Example
Positive	Expresses satisfaction	The road is clean and the air is fresh.
Negative	Expresses dissatisfaction	The factory smoke is bad.
Neutral	Contains mixed or unclear sentiment	The bus is good but the road is dirty.

3. Rule-Based Sentiment Analysis

A simple sentiment analysis system can be developed using positive and negative word dictionaries.

Positive words:

- Good
- Clean
- Fresh
- Helpful
- Excellent
- Safe
- Fast

Negative words:

- Bad
- Dirty
- Smoke
- Pollution
- Dangerous
- Poor
- Harmful

The system checks the words in a complaint and calculates a sentiment score.

The following negation words are also considered:

- Not
- Never
- No

Negation handling is important because the meaning of a word may change depending on the words appearing before it.

Examples:

- "The service is good" – Positive.
- "The service is not good" – Negative.
- "The service is not bad" – Positive.

Sentiment Classification Rules

1. Identify words in the complaint.
2. Compare them with positive and negative dictionaries.
3. Check whether a negation word appears before a sentiment word.
4. Reverse the sentiment contribution when a negation rule applies.
5. Calculate the final sentiment score.

6. Classify the complaint.

Classification:

- Score greater than 0: Positive.
- Score less than 0: Negative.
- Score equal to 0: Neutral.

4. Sentiment Analysis – Pseudocode

```
BEGIN

    INPUT complaint

    SPLIT complaint into words

    INITIALISE sentiment score to zero

    FOR each word in complaint

        CHECK whether word belongs to positive dictionary

        IF positive word is found
            INCREASE sentiment score
        END IF

        CHECK whether word belongs to negative dictionary

        IF negative word is found
            DECREASE sentiment score
        END IF

        CHECK for negation words before the sentiment word
```

```
    IF negation is detected
        REVERSE the sentiment contribution
    END IF

  END FOR

  IF sentiment score is greater than zero
    CLASSIFY as POSITIVE

  ELSE IF sentiment score is less than zero
    CLASSIFY as NEGATIVE

  ELSE
    CLASSIFY as NEUTRAL

  END IF

  DISPLAY sentiment

END
```

5. Extractive Text Summarisation

Text summarisation reduces a long complaint or collection of complaints into a shorter summary.

An extractive summariser selects important sentences from the original text instead of generating entirely new sentences.

The proposed procedure is:

1. Divide the complaint into individual sentences.
2. Remove common stop words.
3. Identify frequently occurring meaningful words.
4. Calculate a score for each sentence based on important words.

5. Select the highest-scoring sentences.
6. Arrange the selected sentences in their original order.
7. Display the summary.

Extractive Summarisation – Pseudocode

```
BEGIN

    INPUT collection of complaints

    SPLIT complaints into sentences

    REMOVE common stop words

    CALCULATE frequency of meaningful words

    FOR each sentence

        CALCULATE sentence importance score

    END FOR

    SELECT sentences with highest scores

    ARRANGE selected sentences in original order

    COMBINE selected sentences

    DISPLAY summary

END
```

6. Rule-Based Citizen Complaint Chatbot

A simple chatbot can be designed to assist citizens in submitting and understanding complaints.

The chatbot can identify keywords and provide responses based on predefined rules.

Proposed chatbot workflow:

1. Accept a complaint from the citizen.
2. Identify important keywords.
3. Determine the complaint category.
4. Identify the station or location if provided.
5. Retrieve relevant information from the database.
6. Provide an appropriate response.
7. Direct serious complaints for human review.

Example:

Citizen: "The factory near my area is releasing harmful smoke."

Chatbot:

"Your complaint appears to be related to air pollution. Please provide the location and additional details so that the issue can be recorded for further review."

7. Alternative AI Approach: Machine Learning

Instead of using a rule-based system, a machine learning model can be trained using previously labelled complaints.

Possible algorithms include:

- Naive Bayes
- Logistic Regression
- Support Vector Machine
- Transformer-based language models

A dataset containing labelled positive, negative, and neutral complaints can be used for training.

The model can be evaluated using accuracy, precision, recall, and F1-score.

8. Comparison of Rule-Based and Machine Learning Approaches

Feature	**Rule-Based Approach**	**Machine Learning Approach**
Implementation	Simple	More complex
Training Data	Not required	Required
Processing	Uses predefined rules	Learns patterns from data
Negation Handling	Requires explicit rules	Can learn from examples
Adaptability	Limited	Can improve with training
Accuracy	Depends on dictionary quality	Depends on data and model quality

9. Limitations

Rule-based sentiment analysis has certain limitations:

- It may not correctly understand sarcasm.
- It may misinterpret double negatives.
- It may fail to understand context.
- Mixed opinions can produce neutral classifications.
- New words may not exist in the predefined dictionaries.

Conclusion:

AI-based techniques can help classify citizen complaints, identify public concerns, and generate short summaries. Rule-based approaches are suitable for simple applications, while machine learning approaches can provide more advanced language understanding when sufficient labelled data is available.

PART B – IMPLEMENTATION AND TESTING

B1 – DATA CLEANING, DATABASE STORAGE AND VISUALISATION

1. Objective

The objective of this task is to implement a complete air-quality data processing pipeline.

The implementation includes:

- Generating and reading the raw CSV dataset.
- Identifying missing and invalid readings.
- Removing duplicate records.
- Converting Fahrenheit into Celsius.
- Applying normalisation.
- Storing cleaned records in SQLite.
- Executing SQL queries.
- Generating daily summaries.
- Creating charts for visual analysis.

2. Dataset Generation

A sample air-quality dataset was generated for 12 monitoring stations.

The dataset contains readings across a 30-day period, with four readings per day for each station.

The raw dataset also includes deliberately introduced data quality problems such as missing values, invalid readings, and duplicate records.

The generated dataset is synthetic and is used for demonstrating the data processing pipeline. It does not represent actual government monitoring measurements.

3. Data Cleaning Process

The raw dataset was processed using the following steps:

1. Read the CSV file.

2. Check the required columns.
3. Identify missing and invalid values.
4. Replace recoverable missing values.
5. Remove duplicate records.
6. Convert Fahrenheit readings into Celsius.
7. Exclude readings greater than 999.
8. Apply Min-Max Normalisation.
9. Save the cleaned dataset.

4. Before and After Cleaning

Before Cleaning

The raw dataset contained:

- Missing PM2.5 values.
- Duplicate records.
- Temperature values reported in Fahrenheit.
- Values requiring validation.

Example raw records:

Station	Date	Hour	PM2.5	PM10	NO	Temperature
Adyar	2026-10-01	0	35	63	10	77°F
Adyar	2026-10-01	6	37	66.6	10	78.35°F
Adyar	2026-10-01	12	18	70.2	10	79.7°F

After Cleaning

The cleaned dataset contained standardised readings.

Station	Date	Hour	PM2.5	Temperature (°C)	Normalised PM2.5
Adyar	2026-10-01	0	35.0	25.00	0.190
Adyar	2026-10-01	6	37.0	25.75	0.222
Adyar	2026-10-01	12	55.4	26.50	0.514
Adyar	2026-10-01	18	-	27.25	0.286

The missing PM2.5 value in the raw dataset was replaced using the selected imputation procedure. The Fahrenheit temperature readings were converted into Celsius.

5. Cleaning Results

The following results were obtained after executing the cleaning pipeline:

Description	Record Count
Total raw records	1441
Duplicate records removed	1
Missing/invalid values replaced	10
Final valid records	1440

The cleaning process successfully reduced the raw dataset to 1440 valid records.

6. SQLite Database Storage

After cleaning, the valid records were inserted into an SQLite database.

The database stores the processed air-quality readings and supports SQL queries for retrieving station-wise and time-based information.

A total of 1440 cleaned records were successfully stored.

7. SQL Query Results

A. Average PM2.5 by Station

Station	Average PM2.5
Adyar	39.25
Ambattur	62.85
Anna Nagar	50.85
Chromepet	59.85
Guindy	41.85
Mylapore	68.85
Perungudi	65.85
Porur	56.85
T Nagar	47.85
Tambaram	53.85
Thiruvanmiyur	71.85
Velachery	44.85

B. Top Five Stations with Highest Average PM2.5

Rank	Station	Average PM2.5
1	Thiruvanmiyur	71.85
2	Mylapore	68.85
3	Perungudi	65.85
4	Ambattur	62.85
5	Chromepet	59.85

Thiruvanmiyur recorded the highest average PM2.5 concentration among the 12 stations, followed by Mylapore and Perungudi.

C. Average PM2.5 by Hour

Hour	Average PM2.5
18:00	58.35
12:00	56.41
06:00	54.35
00:00	52.43

The highest average PM2.5 concentration was recorded at 18:00, followed by 12:00.

8. Daily Summary Generation

A daily summary was generated by grouping the cleaned readings according to station and date.

The summary contains minimum, maximum, average, and count values.

Total daily summary records generated: **360**

This corresponds to 12 monitoring stations over 30 days.

9. Data Visualisation

Four charts were successfully generated.

Chart 1: Monthly PM2.5 Trend

File: monthly_trend.png

A line chart was generated to display PM2.5 variation over the observation period. It helps understand changes in particulate pollution over time.

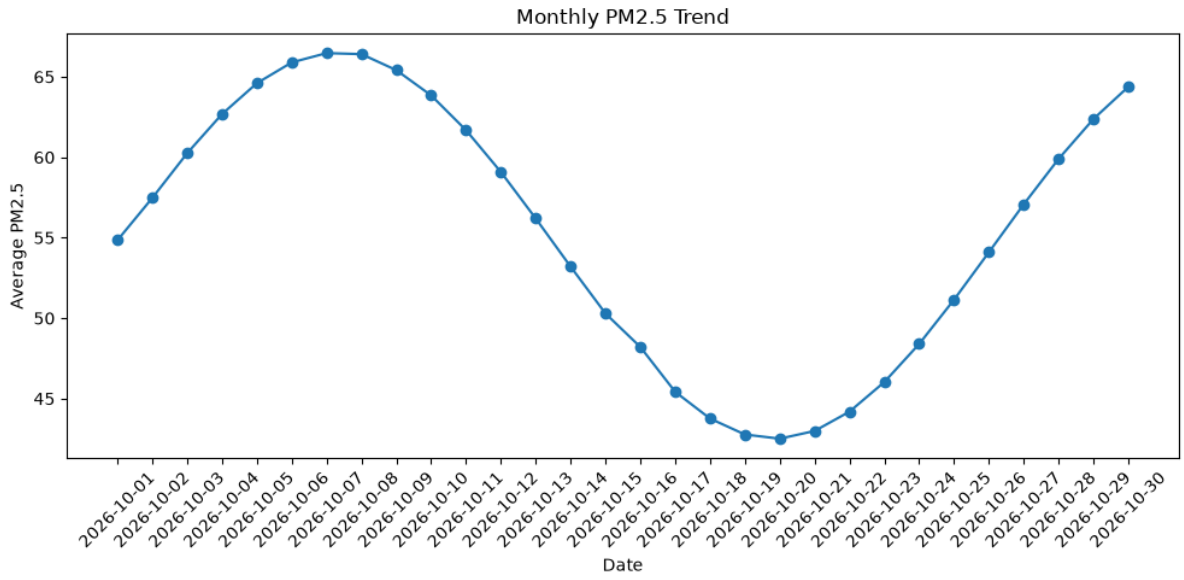


Chart 2: Station Comparison

File: station_comparison.png

A bar chart was generated to compare the average PM2.5 concentration across the 12 monitoring stations. It helps identify stations with higher pollution levels.

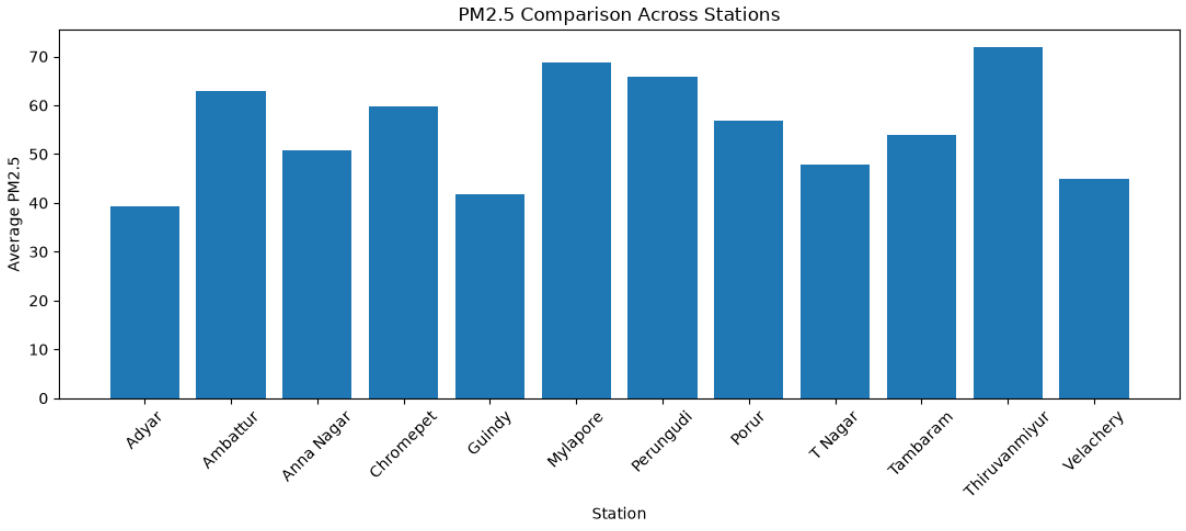


Chart 3: Temperature vs PM2.5

File: temperature_pm25.png

A scatter plot was generated to study the relationship between temperature and PM2.5 concentration. It helps identify possible patterns between the two variables.

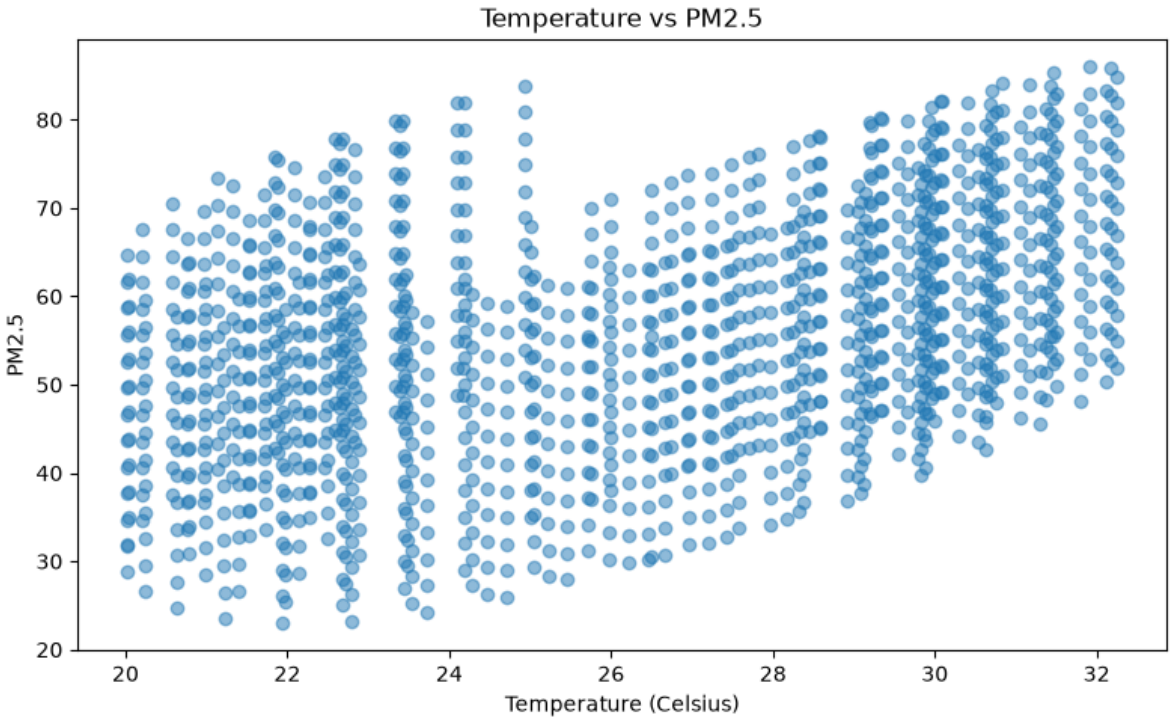
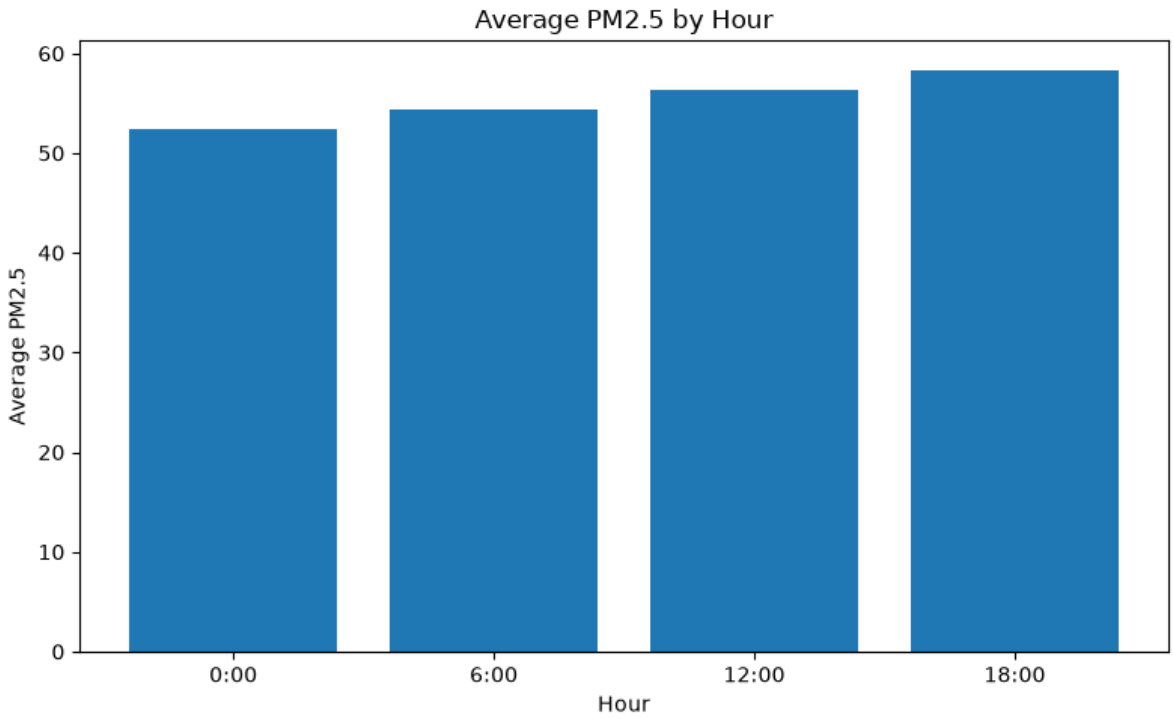


Chart 4: Worst Pollution Hours

File: worst_hours.png

A bar chart was generated to compare average PM2.5 concentrations at different hours. The results show that 18:00 recorded the highest average concentration.



10. Validation Test

A separate test was performed to verify the normalisation procedure and the exclusion of readings greater than 999.

Input values:

48, 95, 121

Expected normalised values:

0.000, 0.644, 1.000

The program produced:

[0.0, 0.644, 1.0]

The test also included a reading greater than 999, which was excluded from the valid records.

11. Implementation Result

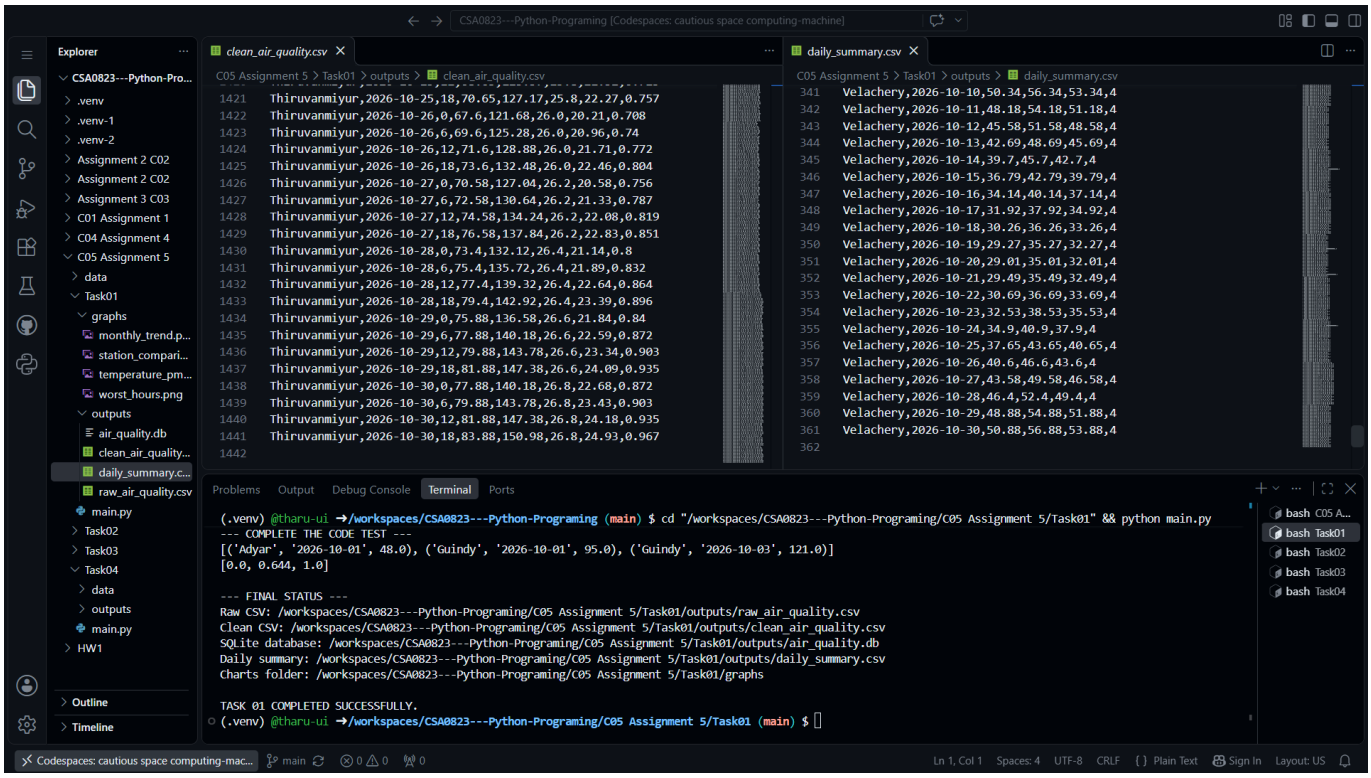
The implementation successfully completed the following operations:

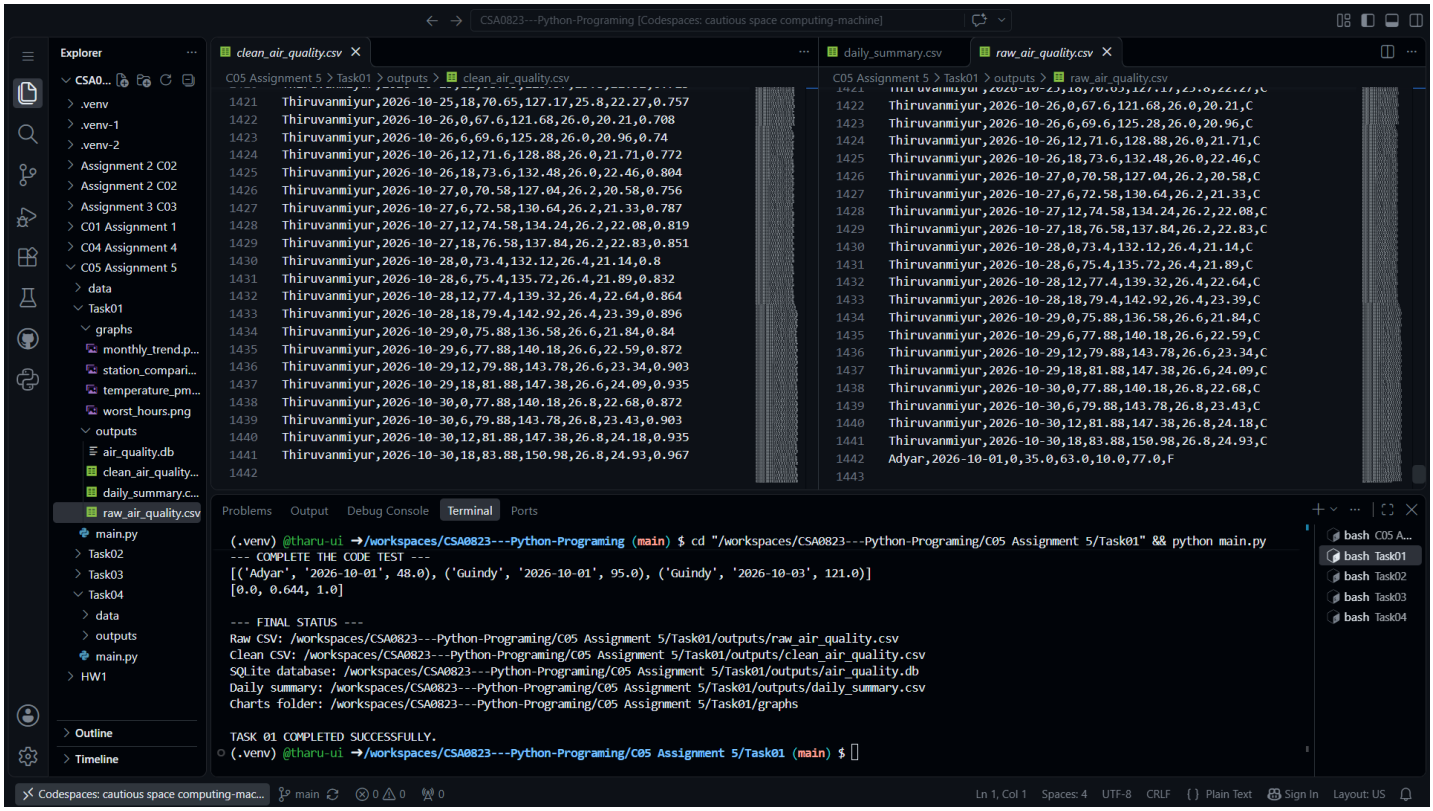
- Generated 1441 raw records.
- Removed one duplicate record.

- Replaced 10 missing or invalid values.
- Produced 1440 valid records.
- Stored the cleaned readings in SQLite.
- Generated 360 daily summary records.
- Executed station-wise and hourly SQL queries.
- Generated four visualisation charts.
- Verified the normalisation and validation procedures.

Conclusion:

The data cleaning and analysis pipeline was implemented successfully. The final dataset was stored in SQLite and analysed using SQL queries and charts. The results demonstrate how raw air-quality readings can be transformed into structured and meaningful information.




CSA0823---Python-Programing [Codespaces: cautious space computing-machine]
Explorer
clean_air_quality.csv
daily_summary.csv
raw_air_quality.csv
C05 Assignment 5 > Task01 > outputs > clean_air_quality.csv
C05 Assignment 5 > Task01 > outputs > raw_air_quality.csv
Problems
Output
Debug Console
Terminal
Ports
(.venv) @tharu-ui ➜/workspaces/CSA0823---Python-Programing (main) $ cd "/workspaces/CSA0823---Python-Programing/C05 Assignment 5/Task01" && python main.py
--- COMPLETE THE CODE TEST ---
[('Adyar', '2026-10-01', 48.0), ('Guindy', '2026-10-01', 95.0), ('Guindy', '2026-10-03', 121.0)]
[0.0, 0.644, 1.0]
--- FINAL STATUS ---
Raw CSV: /workspaces/CSA0823---Python-Programing/C05 Assignment 5/Task01/outputs/raw_air_quality.csv
Clean CSV: /workspaces/CSA0823---Python-Programing/C05 Assignment 5/Task01/outputs/clean_air_quality.csv
SQLite database: /workspaces/CSA0823---Python-Programing/C05 Assignment 5/Task01/outputs/air_quality.db
Daily summary: /workspaces/CSA0823---Python-Programing/C05 Assignment 5/Task01/outputs/daily_summary.csv
Charts folder: /workspaces/CSA0823---Python-Programing/C05 Assignment 5/Task01/graphs
TASK 01 COMPLETED SUCCESSFULLY.
(.venv) @tharu-ui ➜/workspaces/CSA0823---Python-Programing/C05 Assignment 5/Task01 (main) $
Codespaces: cautious space computing-mac...
main
Ln 1, Col 1
Spaces: 4
UTF-8
CRLF
Plain Text
Sign In
Layout: US

B2 – ADVERSARIAL TESTING, SQL INJECTION AND SENTIMENT ANALYSIS

1. Objective

The objective of this task is to test the reliability and security of the data processing system.

The implementation includes:

- Testing malformed CSV files.
- Handling missing columns.
- Detecting incorrect delimiters.
- Handling unsupported text encodings.
- Handling empty files.
- Demonstrating SQL injection.
- Using parameterised SQL queries.
- Testing sentiment analysis with difficult sentences.
- Comparing executemany() and repeated execute() operations.

2. Adversarial CSV Testing

Adversarial testing involves providing unexpected or incorrectly formatted inputs to check whether a program handles errors safely.

Four test files were created.

Test File	**Test Condition**	**Expected Handling**	**Observed Result**
missing_column.csv	Required column is missing	Reject the file	Missing required columns or wrong delimiter
wrong_delimiter.csv	Incorrect CSV delimiter	Detect invalid structure	Missing required columns or wrong delimiter
different_encoding.csv	Unsupported encoding	Display encoding error	Unsupported text encoding

empty.csv	File contains no data	Reject empty input	File is empty

The program successfully detected the malformed inputs and displayed appropriate error messages.

3. SQL Injection Demonstration

SQL injection is a security vulnerability that occurs when untrusted user input is directly combined with an SQL statement.

It may allow an attacker to modify the intended database query.

Unsafe SQL Query

Consider a query created using string concatenation:

SELECT station, pm25

FROM readings

WHERE station = 'user_input';

If user input contains a specially crafted condition, the resulting SQL statement may return unintended records.

In the implementation, the unsafe query returned all available rows:

[('Adyar', 48.0), ('Guindy', 95.0), ('Velachery', 60.0)]

This demonstrates how string concatenation can allow unwanted SQL conditions to affect the query.

4. Prevention Using Parameterised Queries

Parameterised queries separate SQL instructions from user-supplied values.

Instead of directly combining user input with the SQL statement, placeholders are used.

Example:

```
cursor.execute(
    "SELECT station, pm25 FROM readings WHERE station = ?",
    (user_input,)
```

)

The supplied value is treated as data rather than executable SQL.

The safe parameterised query returned:

None

This indicates that the malicious input was treated as a normal value and did not return all database records.

Comparison

Feature	**String Concatenation**	**Parameterised Query**
User Input	Directly combined with SQL	Passed separately
Security	Vulnerable to SQL injection	Helps prevent SQL injection
Query Structure	Can be modified by input	Remains fixed
Recommended	No	Yes

5. Sentiment Analysis with Adversarial Sentences

Sentiment analysis was tested using sentences containing negation, mixed opinions, and sarcasm.

The following results were obtained:

Sentence	**Predicted Sentiment**
The road is clean and the bus is fast.	Positive
The factory smoke is bad.	Negative
The service is not good.	Negative
The service is not bad.	Positive
The roads are not clean.	Negative
The service is never not good.	Negative
The bus is good but the road is dirty.	Neutral

Wonderful! Another day of breathing smoke.	Negative
The staff are helpful but the air is dirty.	Neutral

6. Analysis of Sentiment Results

The system successfully identified several basic positive and negative expressions.

For example:

- "The service is not good" was classified as negative.
- "The service is not bad" was classified as positive.
- "The roads are not clean" was classified as negative.

However, the system has limitations.

The sentence "The service is never not good" was classified as negative because the simple negation rule does not fully understand double negation.

Similarly, sarcasm is difficult to identify using a basic dictionary-based approach. The sentence about breathing smoke was classified as negative because it contained a negative keyword, not because the system truly understood the sarcastic meaning.

Mixed opinions were classified as neutral when positive and negative words appeared together.

7. Comparison of Database Insertion Methods

Two methods were compared for inserting multiple records into SQLite.

Method 1: Repeated execute()

The execute() method is called separately for each record.

Method 2: executemany()

The executemany() method inserts multiple records using a single method call with a collection of values.

Observed execution times:

Method	Execution Time
Repeated execute()	0.030270 seconds
executemany()	0.019250 seconds

The executemany() method completed the insertion faster in the observed test.

It reduces repeated method-call overhead and is more convenient for inserting multiple records.

Both methods still process the records, so the overall insertion work depends on the number of records and database operations.

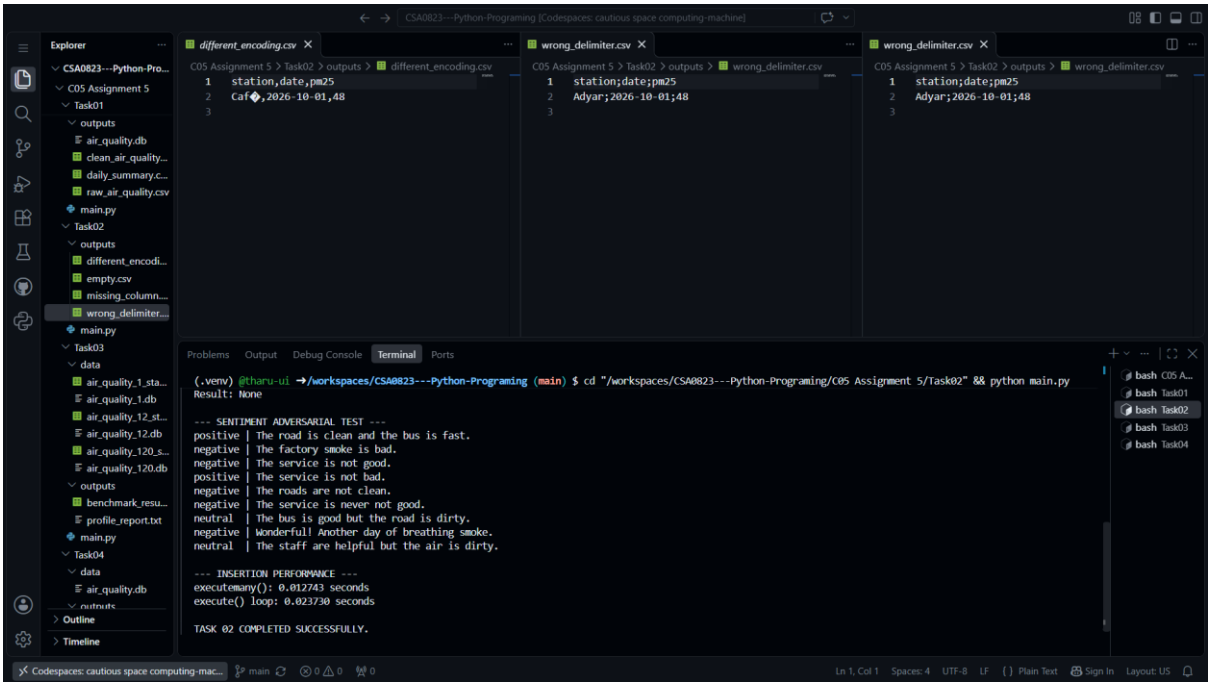
8. Implementation Result

The following operations were completed successfully:

- Tested four malformed CSV files.
- Detected missing columns and incorrect delimiters.
- Detected unsupported encoding.
- Identified empty input files.
- Demonstrated SQL injection using unsafe string concatenation.
- Prevented the demonstrated injection using parameterised SQL.
- Tested nine sentiment examples.
- Compared executemany() and repeated execute().

Conclusion:

The adversarial testing process demonstrated the importance of validating external data and using secure database queries. Parameterised SQL queries provide protection against the demonstrated SQL injection attempt. The sentiment tests also revealed the limitations of simple rule-based language processing.


Explorer
CSA0823---Python-Pro...
C05 Assignment 5
Task01
outputs
air_quality.db
clean_air_quality...
daily_summary.c...
raw_air_quality.csv
main.py
Task02
outputs
different_encodi...
empty.csv
missing_column....
wrong_delimiter....
main.py
Task03
data
air_quality_1_sta...
air_quality_1.db
air_quality_12_st...
air_quality_12.db
air_quality_120_s...
air_quality_120.db
outputs
benchmark_resu...
profile_report.txt
main.py
Task04
data
air_quality.db
Outline
Timeline
different_encoding.csv
C05 Assignment 5 > Task02 > outputs > different_encoding.csv
1 station,date,pm25
2 Caf,2026-10-01,48
3
wrong_delimiter.csv
C05 Assignment 5 > Task02 > outputs > wrong_delimiter.csv
1 station;date;pm25
2 Adyar;2026-10-01;48
3
wrong_delimiter.csv
C05 Assignment 5 > Task02 > outputs > wrong_delimiter.csv
1 station;date;pm25
2 Adyar;2026-10-01;48
3
Problems
Output
Debug Console
Terminal
Ports
(.venv) @tharu-ui →/workspaces/CSA0823---Python-Programing (main) $ cd "/workspaces/CSA0823---Python-Programing/C05 Assignment 5/Task02" && python main.py
Result: None
--- SENTIMENT ADVERSARIAL TEST ---
positive | The road is clean and the bus is fast.
negative | The factory smoke is bad.
negative | The service is not good.
positive | The service is not bad.
negative | The roads are not clean.
negative | The service is never not good.
neutral | The bus is good but the road is dirty.
negative | Wonderful! Another day of breathing smoke.
neutral | The staff are helpful but the air is dirty.
--- INSERTION PERFORMANCE ---
executemany(): 0.012743 seconds
execute() loop: 0.023730 seconds
TASK 02 COMPLETED SUCCESSFULLY.
bash C05 A...
bash Task01
bash Task02
bash Task03
bash Task04
Codespaces: cautious space computing-mac...
main
Ln 1, Col 1
Spaces: 4
UTF-8
LF
Plain Text
Sign In
Layout: US

B3 – PERFORMANCE COMPARISON OF DATA PROCESSING METHODS

1. Objective

The objective of this task is to compare the performance of three data processing methods:

1. Plain Python loops.
2. Pandas groupby().
3. SQLite GROUP BY.

The comparison is performed using datasets of different sizes to understand how each method behaves as the number of records increases.

2. Dataset Sizes

The experiment uses 1, 12, and 120 monitoring stations.

Each station contains 8760 hourly readings representing one year of data.

Number of Stations	Readings per Station	Total Readings
1	8760	8760
12	8760	105120
120	8760	1051200

The daily mean PM2.5 value is calculated for every station and date.

3. Processing Methods

A. Plain Python Loops

The dataset is processed using standard Python iteration.

The program reads the records, groups readings by station and date, and calculates daily mean values.

Advantages:

- Easy to understand.
- Does not require an external data-processing library.
- Provides direct control over processing logic.

Limitations:

- Repeated Python-level operations can become slower for large datasets.
- Additional dictionaries or data structures may be required.

B. Pandas groupby()

Pandas is used to group records by station and date.

The groupby() operation calculates daily mean PM2.5 values.

Advantages:

- Provides convenient data-processing functions.
- Reduces the amount of manual code.
- Performs well for many data-analysis operations.

Limitations:

- Requires an additional library.
- DataFrames may consume considerable memory for large datasets.

C. SQLite GROUP BY

The dataset is stored in SQLite, and SQL aggregation is used to calculate daily means.

Advantages:

- Supports structured data storage.
- Provides reusable SQL queries.
- Can benefit from database optimisation and indexes.

Limitations:

- Requires database storage and query management.
- Database overhead may affect performance depending on dataset size and query structure.

4. Experimental Results

The following execution times were obtained:

Number of Stations	Total Readings	Python Loops (seconds)	Pandas (seconds)	SQL (seconds)
1	8760	0.110301	0.041850	0.008662
12	105120	0.698010	0.092021	0.060316
120	1051200	7.043250	0.664285	0.805105

5. Memory Usage

The peak memory usage measured for the plain Python loop implementation was:

Number of Stations	Total Readings	Python Peak Memory
1	8760	0.26 MB
12	105120	1.36 MB
120	1051200	17.68 MB

The memory values were measured using Python memory-tracking functionality for the loop implementation. They are not a complete memory comparison between all three methods.

6. Verification of Results

All three processing methods were executed using the same datasets.

The calculated daily mean values were compared to verify correctness.

Number of Stations	Python vs Pandas vs SQL Results
1	All three methods agree
12	All three methods agree
120	All three methods agree

All three methods produced equivalent daily summary results.

The number of generated daily summary records was:

Number of Stations	Daily Summary Records
1	365

12	4380
120	43800

7. Performance Analysis

For 1 Station

SQLite recorded the lowest execution time of 0.008662 seconds. Pandas completed the operation in 0.041850 seconds, while plain Python loops required 0.110301 seconds.

For this smaller dataset, SQL performed the fastest.

For 12 Stations

SQLite again recorded the lowest execution time at 0.060316 seconds. Pandas required 0.092021 seconds, while plain Python loops required 0.698010 seconds.

The difference between the plain Python loop method and the other methods became more noticeable as the dataset increased.

For 120 Stations

Pandas completed the operation in 0.664285 seconds, while SQLite required 0.805105 seconds. Plain Python loops required 7.043250 seconds.

For the largest dataset, Pandas performed faster than SQLite and plain Python loops.

8. Overall Comparison

Processing Method	**Small Dataset**	**Medium Dataset**	**Large Dataset**
Python Loops	Slowest	Slowest	Slowest
Pandas	Faster than loops	Faster than loops	Fastest
SQLite	Fastest	Fastest	Faster than loops but slower than Pandas

The results show that no single method is always the fastest.

SQLite performed best for the smaller and medium datasets in this experiment. Pandas performed best for the largest dataset.

Plain Python loops were consistently slower as the dataset size increased.

9. Complexity Analysis

Let n represent the total number of input readings.

Plain Python Loops

Each reading must be processed to calculate the daily summaries.

Time complexity: O(n)

Pandas groupby()

The dataset must be processed and grouped based on station and date.

The overall processing is generally linear or close to linear for this type of aggregation, depending on internal grouping and sorting operations.

SQLite GROUP BY

SQLite processes the relevant records and performs grouping.

The basic aggregation requires processing the records, while additional sorting or indexing operations can affect execution time.

The actual performance depends on the query, database structure, indexes, and dataset size.

10. Profiling Using cProfile

The cProfile module was used to analyse program execution.

The profiler records:

- Number of function calls.
- Total time spent in functions.
- Cumulative execution time.
- Function-level performance information.

The profiling report was successfully generated and saved as:

Task03/outputs/profile_report.txt

The benchmark results were saved as:

Task03/outputs/benchmark_results.csv

These files provide supporting evidence for the performance comparison.

11. Alternative Approach: Running Sums and Counts

An alternative method for calculating daily averages is to maintain a running total and reading count for each station and date.

For every reading:

1. Add the PM2.5 value to the corresponding running total.
2. Increase the reading count.
3. After processing all records, divide the total by the count.

Formula:

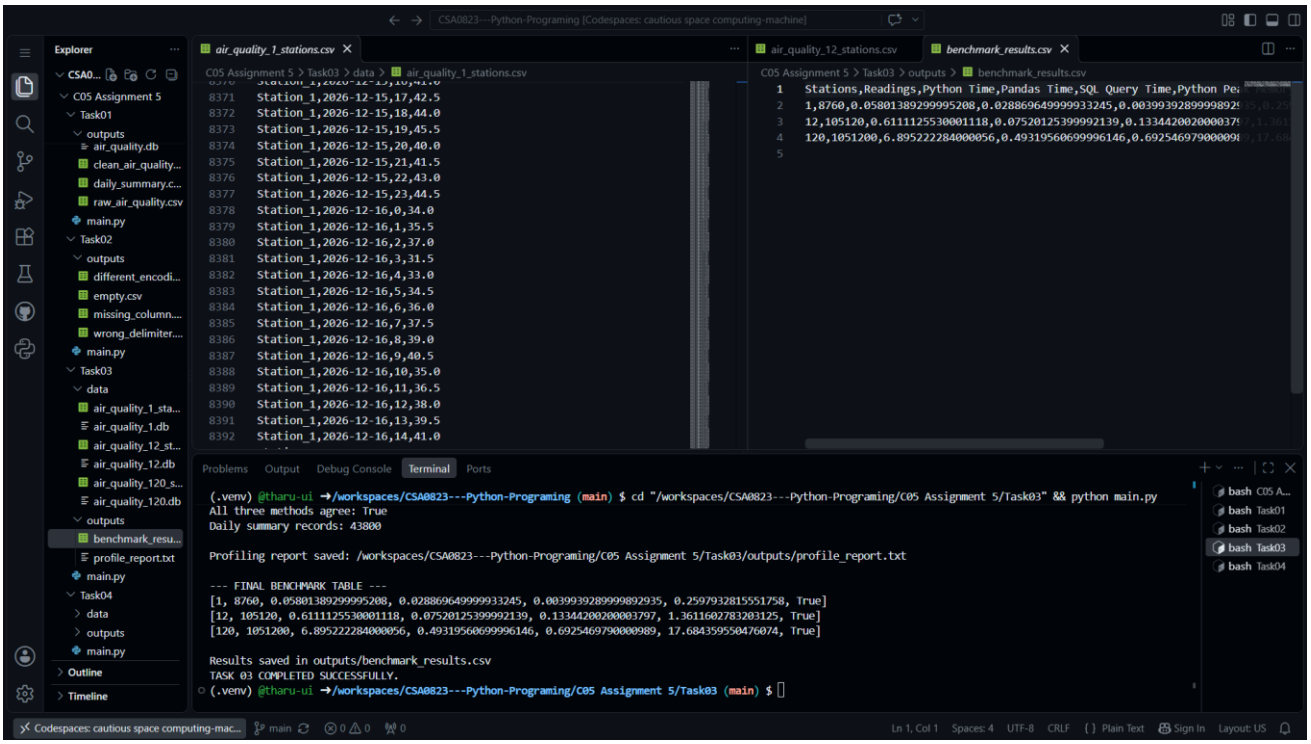
Daily Average = Running Total / Reading Count

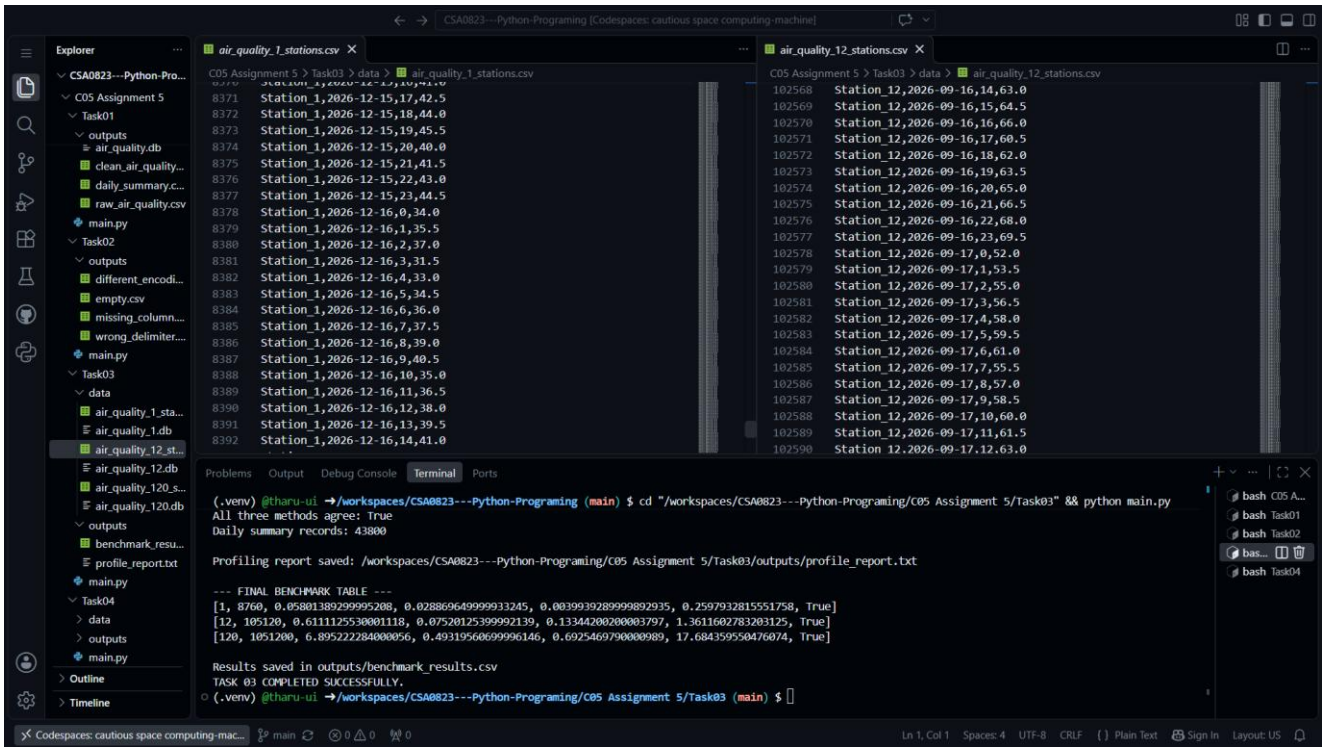
This method avoids storing all readings for a group before calculating its average.

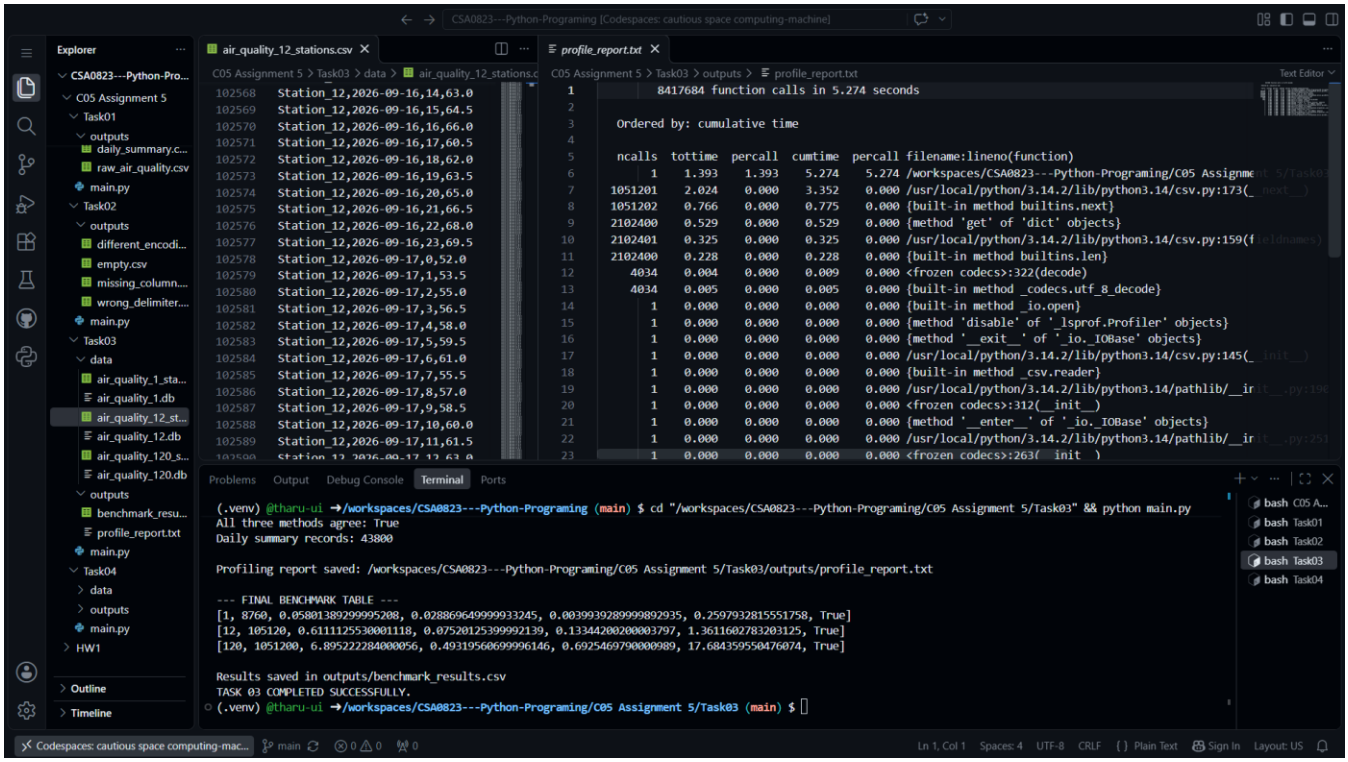
It requires O(n) processing time and can use less intermediate memory when implemented with a suitable dictionary.

Conclusion:

The experiment demonstrated that Pandas and SQLite can provide significant performance improvements over plain Python loops. SQLite performed best for the smaller and medium datasets, while Pandas performed best for the largest dataset in this experiment. All three methods produced matching daily summaries, confirming the correctness of the calculations.


Explorer
air_quality_1_stations.csv
air_quality_12_stations.csv
benchmark_results.csv
C05 Assignment 5 > Task03 > data > air_quality_1_stations.csv
C05 Assignment 5 > Task03 > outputs > benchmark_results.csv
Stations,Readings,Python Time,Pandas Time,SQL Query Time,Python Pe
1,8760,0.0580138929999 5208,0.028869649999933245,0.00399392899998925
12,105120,0.6111125530001118,0.07520125399992139,0.13344200200003797,1.361
120,1051200,6.895222284000056,0.49319560699996146,0.692546979000009
Problems
Output
Debug Console
Terminal
Ports
(.venv) @tharu-ui ➜/workspaces/CSA0823---Python-Programing (main) $ cd "/workspaces/CSA0823---Python-Programing/C05 Assignment 5/Task03" && python main.py
All three methods agree: True
Daily summary records: 43800
Profiling report saved: /workspaces/CSA0823---Python-Programing/C05 Assignment 5/Task03/outputs/profile_report.txt
--- FINAL BENCHMARK TABLE ---
[1, 8760, 0.0580138929999 5208, 0.028869649999933245, 0.00399392899998892935, 0.2597932815551758, True]
[12, 105120, 0.6111125530001118, 0.07520125399992139, 0.13344200200003797, 1.3611602783203125, True]
[120, 1051200, 6.895222284000056, 0.49319560699996146, 0.6925469790000989, 17.684359550476074, True]
Results saved in outputs/benchmark_results.csv
TASK 03 COMPLETED SUCCESSFULLY.
(.venv) @tharu-ui ➜/workspaces/CSA0823---Python-Programing/C05 Assignment 5/Task03 (main) $
Codespaces: cautious space computing-mac...
main
Ln 1, Col 1
Spaces: 4
UTF-8
CRLF
Plain Text
Sign In
Layout: US


Explorer
air_quality_1_stations.csv
air_quality_12_stations.csv
C05 Assignment 5 > Task03 > data > air_quality_1_stations.csv
C05 Assignment 5 > Task03 > data > air_quality_12_stations.csv
Problems
Output
Debug Console
Terminal
Ports
(.venv) @tharu-ui ➜/workspaces/CSA0823---Python-Programing (main) $ cd "/workspaces/CSA0823---Python-Programing/C05 Assignment 5/Task03" && python main.py
All three methods agree: True
Daily summary records: 43800
Profiling report saved: /workspaces/CSA0823---Python-Programing/C05 Assignment 5/Task03/outputs/profile_report.txt
--- FINAL BENCHMARK TABLE ---
[1, 8760, 0.0580138929999 5208, 0.028869649999933245, 0.00399392899998892935, 0.2597932815551758, True]
[12, 105120, 0.6111125530001118, 0.07520125399992139, 0.13344200200003797, 1.3611602783203125, True]
[120, 1051200, 6.895222284000056, 0.49319560699996146, 0.6925469790000989, 17.684359550476074, True]
Results saved in outputs/benchmark_results.csv
TASK 03 COMPLETED SUCCESSFULLY.
(.venv) @tharu-ui ➜/workspaces/CSA0823---Python-Programing/C05 Assignment 5/Task03 (main) $
Codespaces: cautious space computing-mac...
main
Ln 1, Col 1
Spaces: 4
UTF-8
CRLF
Plain Text
Sign In
Layout: US


CSA0823---Python-Programing [Codespaces: cautious space computing-machine]
Explorer
CSA0823---Python-Pro...
C05 Assignment 5
Task01
outputs
daily_summary.c...
raw_air_quality.csv
main.py
Task02
outputs
different_encodi...
empty.csv
missing_column....
wrong_delimiter....
main.py
Task03
data
air_quality_1_sta...
air_quality_1.db
air_quality_12_st...
air_quality_12.db
air_quality_120_s...
air_quality_120.db
outputs
benchmark_resu...
profile_report.txt
main.py
Task04
data
outputs
main.py
HW1
Outline
Timeline
air_quality_12_stations.csv
C05 Assignment 5 > Task03 > data > air_quality_12_stations.c
102568 Station_12,2026-09-16,14,63.0
102569 Station_12,2026-09-16,15,64.5
102570 Station_12,2026-09-16,16,66.0
102571 Station_12,2026-09-16,17,60.5
102572 Station_12,2026-09-16,18,62.0
102573 Station_12,2026-09-16,19,63.5
102574 Station_12,2026-09-16,20,65.0
102575 Station_12,2026-09-16,21,66.5
102576 Station_12,2026-09-16,22,68.0
102577 Station_12,2026-09-16,23,69.5
102578 Station_12,2026-09-17,0,52.0
102579 Station_12,2026-09-17,1,53.5
102580 Station_12,2026-09-17,2,55.0
102581 Station_12,2026-09-17,3,56.5
102582 Station_12,2026-09-17,4,58.0
102583 Station_12,2026-09-17,5,59.5
102584 Station_12,2026-09-17,6,61.0
102585 Station_12,2026-09-17,7,55.5
102586 Station_12,2026-09-17,8,57.0
102587 Station_12,2026-09-17,9,58.5
102588 Station_12,2026-09-17,10,60.0
102589 Station_12,2026-09-17,11,61.5
profile_report.txt
C05 Assignment 5 > Task03 > outputs > profile_report.txt
Text Editor
8417684 function calls in 5.274 seconds
Ordered by: cumulative time
ncalls tottime percall cumtime percall filename:lineno(function)
1 1.393 1.393 5.274 5.274 /workspaces/CSA0823---Python-Programing/C05 Assignme
1051201 2.024 0.000 3.352 0.000 /usr/local/python/3.14.2/lib/python3.14/csv.py:173(_
1051202 0.766 0.000 0.775 0.000 {built-in method builtins.next}
2102400 0.529 0.000 0.529 0.000 {method 'get' of 'dict' objects}
2102401 0.325 0.000 0.325 0.000 /usr/local/python/3.14.2/lib/python3.14/csv.py:159(f
2102400 0.228 0.000 0.228 0.000 {built-in method builtins.len}
4034 0.004 0.000 0.009 0.000 <frozen codecs>:322(decode)
4034 0.005 0.000 0.005 0.000 {built-in method _codecs.utf_8_decode}
1 0.000 0.000 0.000 0.000 {built-in method _io.open}
1 0.000 0.000 0.000 0.000 {method 'disable' of '_lsprof.Profiler' objects}
1 0.000 0.000 0.000 0.000 {method '__exit__' of '_io._IOBase' objects}
1 0.000 0.000 0.000 0.000 /usr/local/python/3.14.2/lib/python3.14/csv.py:145(_
1 0.000 0.000 0.000 0.000 {built-in method _csv.reader}
1 0.000 0.000 0.000 0.000 /usr/local/python/3.14.2/lib/python3.14/pathlib/__ir
1 0.000 0.000 0.000 0.000 <frozen codecs>:312(__init__)
1 0.000 0.000 0.000 0.000 {method '__enter__' of '_io._IOBase' objects}
1 0.000 0.000 0.000 0.000 /usr/local/python/3.14.2/lib/python3.14/pathlib/__ir
1 0.000 0.000 0.000 0.000 <frozen codecs>:263(__init__)
Problems
Output
Debug Console
Terminal
Ports
(.venv) @tharu-ui ➜/workspaces/CSA0823---Python-Programing (main) $ cd "/workspaces/CSA0823---Python-Programing/C05 Assignment 5/Task03" && python main.py
All three methods agree: True
Daily summary records: 43800
Profiling report saved: /workspaces/CSA0823---Python-Programing/C05 Assignment 5/Task03/outputs/profile_report.txt
--- FINAL BENCHMARK TABLE ---
[1, 8760, 0.05801389299995208, 0.028869649999933245, 0.0039939289999892935, 0.2597932815551758, True]
[12, 105120, 0.6111125530001118, 0.07520125399992139, 0.13344200200003797, 1.361160278320312 5, True]
[120, 1051200, 6.895222284000056, 0.4931956069996146, 0.6925469790000989, 17.684359550476074, True]
Results saved in outputs/benchmark_results.csv
TASK 03 COMPLETED SUCCESSFULLY.
(.venv) @tharu-ui ➜/workspaces/CSA0823---Python-Programing/C05 Assignment 5/Task03 (main) $
bash C05 A...
bash Task01
bash Task02
bash Task03
bash Task04
Codespaces: cautious space computing-mac...
main
Ln 1, Col 1
Spaces: 4
UTF-8
CRLF
Plain Text
Sign In
Layout: US

B4 – SENTIMENT ANALYSIS, TEXT SUMMARISATION AND DATABASE INDEXING

1. Objective

The objective of this task is to apply text analysis techniques to citizen complaints and evaluate the performance improvement obtained through SQLite indexing.

The implementation includes:

- Analysing 50 citizen complaints.
- Classifying complaints into positive, negative, and neutral categories.
- Comparing predicted sentiment with manually assigned actual labels.
- Generating an extractive summary.
- Creating a database index on station and date.
- Measuring query execution time before and after indexing.
- Examining the database query execution plan.

2. Sentiment Analysis of Citizen Complaints

A collection of 50 complaints was processed using a rule-based sentiment analysis method.

The system identifies positive and negative words and uses simple negation rules to determine the sentiment of each complaint.

The predicted sentiment is compared with manually assigned actual labels.

3. Sentiment Classification Results

The following results were obtained:

Evaluation Metric	**Result**
Total complaints	50
Correct predictions	49
Incorrect predictions	1
Agreement	98.00%

Agreement was calculated as:

Agreement = (Correct Predictions / Total Complaints) × 100

Agreement = (49 / 50) × 100

Agreement = 98%

Therefore, the system achieved 98.00% agreement with the manually assigned labels.

4. Incorrect Classification

One complaint was incorrectly classified.

Complaint	Actual Sentiment	Predicted Sentiment
The pollution problem is serious.	Negative	Neutral

The complaint was classified as neutral because the simple word dictionary did not sufficiently recognise the word "serious" as a negative expression.

This demonstrates that dictionary-based sentiment analysis depends heavily on the words included in its predefined vocabulary.

5. Analysis of Results

The system correctly classified 49 out of 50 complaints.

The results show that a simple rule-based approach can identify many basic positive and negative expressions.

However, its performance depends on the quality of the sentiment dictionary and the rules used for handling negation.

A more advanced system could include additional environmental terms and a machine learning model trained using labelled complaint data.

The 98% agreement represents comparison with the manually assigned labels used in this experiment. It should not be interpreted as a guarantee of performance on all real-world complaints.

6. Extractive Text Summarisation

An extractive summariser was used to select important sentences from the complaint collection.

The summariser identifies meaningful words, calculates sentence importance, and selects sentences that represent the main information.

The generated summary was:

"The roads are clean and the air is fresh. The factory smoke makes the air bad. The bus service is fast and helpful. The factory releases dirty smoke. The air is fresh and clean."

The summary provides a shorter representation of the original complaint collection.

It highlights important observations about road cleanliness, air quality, factory smoke, and bus services.

7. SQLite Database Indexing

Database indexing improves the speed of data retrieval by creating an additional data structure that helps locate records more efficiently.

For this task, a composite index was created on the station and day columns.

Index creation:

CREATE INDEX idx_station_day

ON readings(station, day);

The index supports queries that filter records using both station and date.

8. Query Performance Before and After Indexing

The database contained 105120 records.

The same filtered query was executed before and after creating the index.

Observed results:

Query Condition	**Execution Time**
Without Index	7.255076 seconds
With Index	0.020597 seconds

The query execution time was significantly reduced after indexing.

9. Calculation of Performance Improvement

Percentage reduction in execution time:

Reduction = ((Time Before - Time After) / Time Before) × 100

Substituting the observed values:

Reduction = ((7.255076 - 0.020597) / 7.255076) × 100

Reduction = 99.72%

Therefore, indexing reduced the observed query execution time by approximately 99.72%.

10. Query Execution Plan

The SQLite query execution plan returned:

(4, 0, 0, 'SEARCH readings USING INDEX idx_station_day (station=? AND day=?)')

This output indicates that SQLite used the created index to search for records matching the station and day conditions.

The query plan confirms that the index was used instead of performing the same unrestricted search through the table.

11. Importance of Indexing

The comparison demonstrates the importance of indexing in database applications.

Without an index, the database may need to examine a large number of records to identify matching rows.

With an appropriate index, the database can locate matching records more efficiently.

The improvement is especially useful when the database contains a large number of records and similar filtered queries are executed repeatedly.

However, indexes also require additional storage and may increase the cost of insert, update, and delete operations because the index must be maintained.

12. Production Decision

For a production air-quality monitoring system, a composite index on station and date is useful because users may frequently request readings for a particular station on a particular day.

The observed execution time decreased from 7.255076 seconds to 0.020597 seconds after indexing, representing a reduction of approximately 99.72%.

Scaling to 100 Additional Monitoring Stations

If 100 additional monitoring stations generate readings every 15 minutes, each station produces 96 readings per day.

Total additional readings per day:

$100 \times 96 = 9{,}600$ readings

Total additional readings over 30 days:

$9{,}600 \times 30 = 288{,}000$ readings

As the number of readings increases, database indexing becomes important for faster retrieval. The composite index on station and day can support frequently executed station-wise and date-wise queries.

Handling 50,000 Complaints per Month

For approximately 50,000 citizen complaints per month, sentiment analysis can be performed in batches. The complaint records and their predicted sentiment labels can be stored in a database for further retrieval and analysis.

A larger database management system may be considered if concurrent users, storage requirements, and processing demands increase significantly.

Therefore, SQLite is suitable for the current experimental application, while a scalable database architecture and batch-processing approach may be considered for a larger production system.

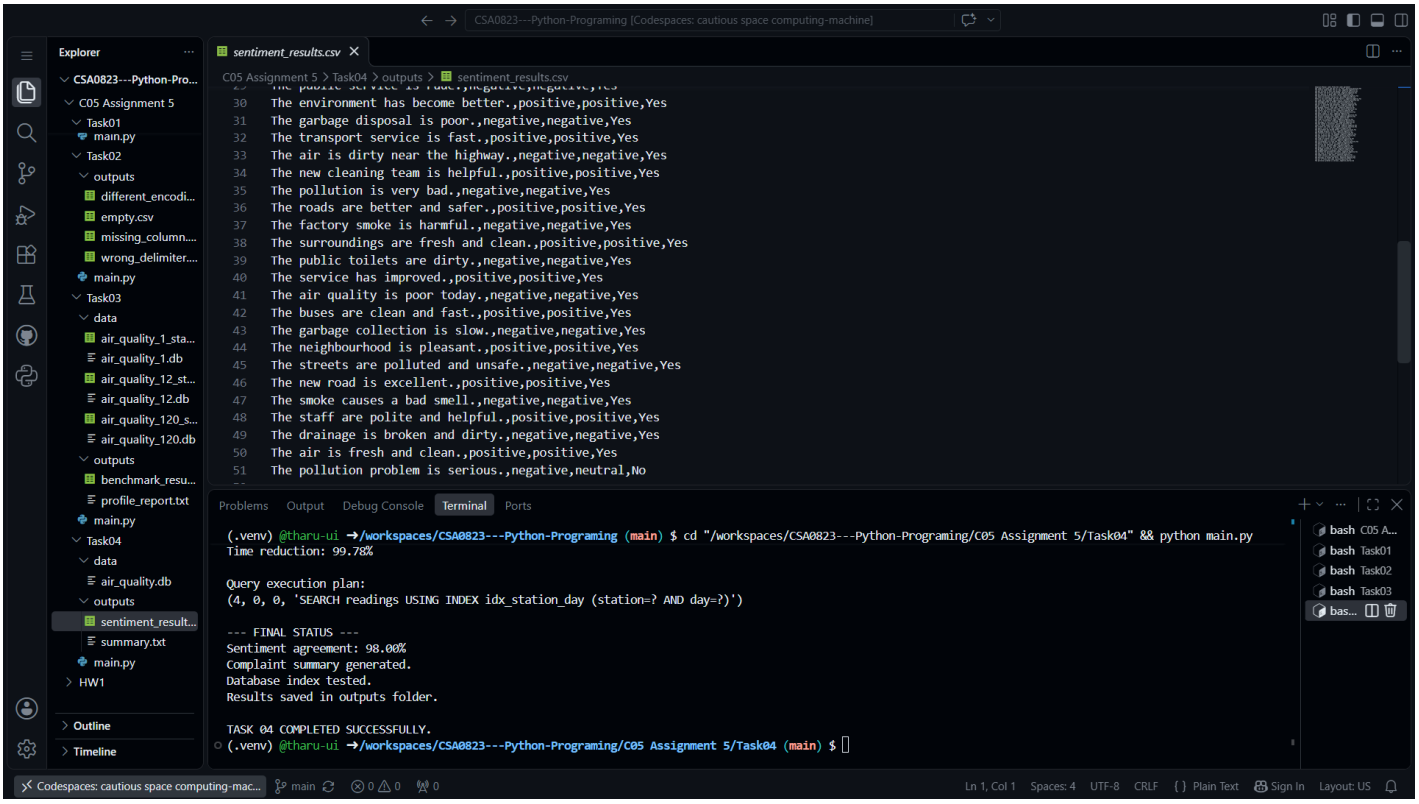
13. Implementation Result

The following operations were completed successfully:

- Processed 50 citizen complaints.
- Compared predicted and actual sentiment labels.
- Obtained 49 correct predictions.
- Achieved 98.00% agreement.
- Identified one incorrectly classified complaint.
- Generated an extractive summary.
- Stored 105120 records in the SQLite database.
- Executed the filtered query without an index.
- Created a composite index on station and day.
- Executed the query again using the index.
- Verified index usage through the query execution plan.
- Obtained a 99.72% reduction in observed query execution time.

Conclusion:

The implementation demonstrated how sentiment analysis and extractive summarisation can help process citizen complaints. Database indexing also significantly improved the observed query performance. These techniques can support faster retrieval and better analysis in an air-quality monitoring application.


CSA0823---Python-Programing [Codespaces: cautious space computing-machine]
Explorer
CSA0823---Python-Pro...
C05 Assignment 5
Task01
main.py
Task02
outputs
different_encodi...
empty.csv
missing_column....
wrong_delimiter....
main.py
Task03
data
air_quality_1_sta...
air_quality_1.db
air_quality_12_st...
air_quality_12.db
air_quality_120_s...
air_quality_120.db
outputs
benchmark_resu...
profile_report.txt
main.py
Task04
data
air_quality.db
outputs
sentiment_result...
summary.txt
main.py
HW1
Outline
Timeline
sentiment_results.csv
C05 Assignment 5 > Task04 > outputs > sentiment_results.csv
30 The environment has become better.,positive,positive,Yes
31 The garbage disposal is poor.,negative,negative,Yes
32 The transport service is fast.,positive,positive,Yes
33 The air is dirty near the highway.,negative,negative,Yes
34 The new cleaning team is helpful.,positive,positive,Yes
35 The pollution is very bad.,negative,negative,Yes
36 The roads are better and safer.,positive,positive,Yes
37 The factory smoke is harmful.,negative,negative,Yes
38 The surroundings are fresh and clean.,positive,positive,Yes
39 The public toilets are dirty.,negative,negative,Yes
40 The service has improved.,positive,positive,Yes
41 The air quality is poor today.,negative,negative,Yes
42 The buses are clean and fast.,positive,positive,Yes
43 The garbage collection is slow.,negative,negative,Yes
44 The neighbourhood is pleasant.,positive,positive,Yes
45 The streets are polluted and unsafe.,negative,negative,Yes
46 The new road is excellent.,positive,positive,Yes
47 The smoke causes a bad smell.,negative,negative,Yes
48 The staff are polite and helpful.,positive,positive,Yes
49 The drainage is broken and dirty.,negative,negative,Yes
50 The air is fresh and clean.,positive,positive,Yes
51 The pollution problem is serious.,negative,neutral,No
Problems
Output
Debug Console
Terminal
Ports
(.venv) @tharu-ui ➜/workspaces/CSA0823---Python-Programing (main) $ cd "/workspaces/CSA0823---Python-Programing/C05 Assignment 5/Task04" && python main.py
Time reduction: 99.78%
Query execution plan:
(4, 0, 0, 'SEARCH readings USING INDEX idx_station_day (station=? AND day=?)')
--- FINAL STATUS ---
Sentiment agreement: 98.00%
Complaint summary generated.
Database index tested.
Results saved in outputs folder.
TASK 04 COMPLETED SUCCESSFULLY.
(.venv) @tharu-ui ➜/workspaces/CSA0823---Python-Programing/C05 Assignment 5/Task04 (main) $
bash C05 A...
bash Task01
bash Task02
bash Task03
bas...
Codespaces: cautious space computing-mac...
main
Ln 1, Col 1
Spaces: 4
UTF-8
CRLF
Plain Text
Sign In
Layout: US